

Proyecto 1	Reproductor de música
Curso	Estructura de datos
Docente	Jacqueline Aldridge
Fecha	Martes, 08 de septiembre de 2026
Fecha de entrega	Viernes 02 de octubre 2026, 12:00 (medio día)
Fecha de presentación	Martes 06 de octubre 14.30 - 16.00 Viernes 09 de octubre 11.20 - 12.50
Modalidad	Grupos de 3 estudiantes
Lenguaje	C (estándar C11 o C99)
Ponderación	50% de la evaluación 1 (30%)

Descripción del proyecto

En este proyecto, los estudiantes deberán diseñar e implementar un sistema que simule el funcionamiento de un reproductor de música, permitiendo generar, almacenar, ordenar y buscar información de canciones.

El sistema deberá incluir la generación automática de datos, creando un catálogo de canciones con los atributos *ID*, *título*, *artista*, *álbum*, *género*, *año*, *duración* y *cantidad_de_reproducciones*. A partir de este catálogo, el sistema deberá permitir operaciones propias de un reproductor real, como reproducir canciones, manipular la lista de reproducción y mantener un historial de canciones escuchadas.

Todo el sistema debe implementarse exclusivamente con arreglos.

Nota: Este proyecto corresponde a una simulación de un reproductor de música y no requiere la reproducción real de archivos de audio. Cuando se indique reproducir una canción, se entenderá que el sistema debe mostrar la información de la canción en la terminal y actualizar los datos correspondientes, como la cantidad de reproducciones y el historial.

Objetivos de aprendizaje

- Implementar y manipular eficientemente arreglos para el almacenamiento, acceso y gestión del catálogo musical.
- Implementar correctamente algoritmos de ordenamiento y búsqueda.
- Documentar adecuadamente el código y las decisiones de diseño del sistema.

Aplicación práctica

Generación y carga de los datos

Toda la información con la que trabaja el sistema proviene de un catálogo de canciones generado de forma aleatoria el cual debe tener un tamaño suficiente para que su funcionamiento sea representativo.

El catálogo debe contener las siguientes columnas:

- **id** (valor entero único y autoincremental)
- **título** (generado aleatoriamente)
- **artista** (seleccionado desde un arreglo)
- **album** (generado aleatoriamente)
- **genero** (seleccionado desde un arreglo)
- **duracion_seg** (número aleatorio)
- **año** (número aleatorio)

- **n_reproducciones** (número aleatorio)

Los campos de texto generados «aleatoriamente» (por ejemplo, título y álbum) podrán construirse mediante la combinación de caracteres seleccionados al azar, aunque también se podrá optar por formar los valores a partir de palabras almacenadas en un arreglo o archivo, o seleccionarlos directamente desde una lista amplia de opciones predefinidas. Un ejemplo utilizando diferentes metodologías de generación de datos se presenta a continuación.

```
id,título,artista,album,genero,duracion_seg,año,n_reproducciones
1,Aquiesce,Oasis,The Masterplan,Rock,288,1998,18320
2,Hood,Trueno,Bien o Mal,Hip-Hop,178,2022,8730
3,Amigo,Las ardillas azules,No,Funk,195,2026,9992
4,ymXaWSdA,Foo Fighters,aETgcQwSl_,Rock,205,2025,6503
```

Además, antes de almacenar la información del catálogo, se deberá verificar que los valores numéricos sean consistentes con valores esperables en un contexto real. Por ejemplo, la duración y la cantidad de reproducciones no podrán ser negativas, mientras que el año deberá encontrarse dentro de un rango razonable.

Luego de generar los datos, se deberá implementar un algoritmo que permita mezclar aleatoriamente los registros, por ejemplo, mediante el algoritmo de Fisher-Yates. Posteriormente, los datos deberán almacenarse en un archivo CSV. Este archivo deberá generarse una única vez y, en las ejecuciones posteriores, deberá ser leído por el reproductor para cargar el catálogo de canciones.

Funcionalidades del sistema

El sistema debe contar con las siguientes funcionalidades:

1. Gestión de catalogo:

- Generar aleatoriamente un catálogo de N canciones.
- Listar los artistas disponibles.
- Obtener canciones por género musical:
 - Obtener cantidad de canciones por género musical.
 - Listar canciones de un género musical ingresado por teclado.
 - Obtener cantidad de canciones usando un género ingresado por teclado.

2. Ordenamiento y búsqueda:

- Ordenar el catálogo utilizando todos los campos disponibles.
- Buscar canciones por ID, nombre o artista
- Obtener todas las canciones de un artista ingresado por consola.

3. Ranking y estadísticas:

- Mostrar un ranking de las n canciones más escuchadas.
- Obtener la canción más escuchada por artista o por género.

4. Fila de reproducción:

- Consultar la fila de reproducción.
- Añadir una canción al comienzo de la fila de reproducción.
- Quitar una canción de la fila de reproducción por posición o por ID.
- Vaciar completamente la fila de reproducción.
- Detectar posibles duplicados al añadir canciones, evitando que una misma canción sea agregada más de una vez a la fila de reproducción.

5. Reproducción e historial:

- Consultar el historial de reproducción.
 - El historial debe almacenar únicamente las últimas K canciones reproducidas. Al superar este límite, se debe descartar la canción más antigua y desplazar las restantes para incorporar la nueva.
- Reproducir la primera canción de la fila de reproducción. Al reproducirla, se debe incrementar su valor de $n_reproducciones$ y agregarla al historial.

6. **Exportación:**

- Exportar el catálogo actualizado a un nuevo archivo CSV después de modificar los valores de $n_reproducciones$.

7. **Interfaz:**

- Proporcionar un menú de línea de comandos que permita interactuar con todas las funcionalidades del sistema.

Deberán implementar los algoritmos y estructuras de datos necesarios para el correcto funcionamiento de todas las funcionalidades solicitadas. Además, será obligatorio implementar los siguientes algoritmos:

1. Un algoritmo de ordenamiento iterativo: **Bubble sort, Insertion sort, Selection sort**
2. Un algoritmo de ordenamiento basado en recursividad: **Merge Sort o Quick Sort**
3. **Búsqueda binaria** en su versión recursiva.

Normativas de codificación en C

- Deben usar nombres de variables y funciones descriptivas, además de usar un esquema de nomenclatura coherente (por ejemplo snake_case o CamelCase).
- Deben mantener la consistencia en la indentación y espaciado.
- Deben dividir el código en funciones pequeñas y manejables que realicen una única tarea.
- Deben identificar cada código con el programador.
- Deben incluir comentarios claros y precisos para explicar la funcionalidad de bloques de código complejos
- Deben documentar cada función con comentarios que expliquen su propósito, parámetros de entrada, valores de retorno y posibles errores.
- Deben implementar mecanismo robustos para la gestión de errores, asegurándose de que todos los errores posibles sean capturados y manejados adecuadamente.
- Deben evitar el uso innecesario de variables globales.
- Deben evitar fugas de memoria liberando memoria asignada dinámicamente.

Entregables

- Repositorio de GitHub con:
 - Código fuente: Archivos .c y .h
 - Makefile
 - README del proyecto
 - README de buenas practicas y organización de grupo
 - Informe: Documento PDF de máximo 15 hojas que incluya:
 - Descripción de la implementación de los algoritmos y estructuras utilizadas
 - Contribución por estudiante y declaración de IA: Se debe detallar brevemente el aporte de cada estudiante al trabajo final

Declaración de IA

Cada grupo debe incluir dentro de su informe una declaración de uso de IA. En ella cada estudiante debe indicar si utilizó o no herramientas de inteligencia artificial durante el desarrollo del proyecto, y en caso afirmativo, especificar cuáles y para qué las usó: si fue para resolver dudas conceptuales, para depurar errores, para revisar o mejorar código ya escrito, o para cualquier otro fin.

Si parte del código entregado fue generado directamente por una IA, esto debe quedar explícitamente señalado en la declaración, indicando qué función o sección corresponde y cómo fue adaptada o comprendida por el estudiante.

El uso de herramientas de inteligencia artificial (IA) está permitido en el apoyo del desarrollo de la tarea y en la resolución de errores, pero no se permite la entrega de tareas totalmente generadas con IA.

Consideraciones generales

- Por cada 12hrs de retraso en la entrega del proyecto, se descontarán 5 décimas.
- Para que un proyecto sea evaluado los estudiantes deberán:
 - entregar el repositorio con todo lo solicitado en la sección de **Entregables**
 - realizar la presentación
 - informar contribuciones de cada estudiante y uso de IA
- En caso de que un estudiante no asista a la presentación, el resto del grupo deberá presentar de todas formas, y el estudiante ausente será calificado con nota mínima.
- Las presentaciones deberán realizarse los días 6 y 9 de octubre y no podrán superar los 10 minutos de exposición. El orden de presentación será designado por la profesora. En caso de que un grupo no se encuentre presente a la hora de ser llamados se penalizará con 5 décimas.
- Todos los integrantes del grupo deberán aportar tanto en la escritura del informe como en la implementación del código y en la presentación. En caso contrario, existirá una penalización.
- El proyecto debe incluir un *Makefile* que permita compilar y ejecutar el programa en un entorno Linux.
- La presentación debe ser un fiel reflejo de la información proporcionada en el repositorio y en el informe, por lo que no se admitirán presentaciones con contenido adicional.
- La conformación de grupos deberá ser informada mediante correo electrónico a más tardar el jueves 10 de septiembre a las 23.59.
- El *README* de buenas practicas y organización del grupo deberá subirse al repositorio a más tardar el 22/09/2026.

Evaluación

La evaluación del proyecto se realizará considerando tres componentes principales:

- **Evaluación del código - 65%**
- **Evaluación de informe - 25%**
- **Evaluación de pares - 10%**

Estos tres componentes determinan la **nota base** del proyecto. Adicionalmente, la **presentación** se evaluará de forma independiente y su nota actuará como un **factor de ponderación** sobre la nota base, en el rango de 0.5 a 1.1.

Rúbrica de evaluación código

Criterio	Descripción	Peso (%)
Funcionalidad	Se evalúa si el proyecto cumple con todos los requisitos funcionales, asegurando que las estructuras de datos solicitadas estén correctamente implementadas y que el sistema se comporte adecuadamente bajo diferentes escenarios, incluidos los casos límite.	45 %
Estructura y Organización	Valora la claridad en la organización del código, su modularidad y cómo las funciones y dependencias están gestionadas. El código debe ser fácil de leer, mantener y modificar, mostrando una estructura lógica y eficiente. Además, se evaluará el uso adecuado de Git, considerando un historial de commits que refleje el desarrollo progresivo del proyecto.	20 %
Calidad del Código	Analiza la calidad general del código fuente, prestando atención a las buenas prácticas de programación, la legibilidad (nombres de variables descriptivos, comentarios), la eficiencia de los algoritmos y la ausencia de código redundante.	15 %
Documentación	Se valora la calidad de la documentación interna y externa. Esto incluye comentarios útiles en el código y documentos que expliquen el diseño, la lógica detrás del código y cómo usar el proyecto. La documentación debe permitir que cualquier desarrollador comprenda el proyecto sin dificultad.	10 %
Pruebas y Verificación	Se evalúa la exhaustividad y efectividad de las pruebas realizadas para asegurar que el proyecto funcione correctamente. Esto incluye pruebas con un enfoque en detectar posibles errores y manejar casos límite.	10 %

Rúbrica de evaluación informe

Criterio	Descripción	Peso (%)
Claridad y Organización	Estructura coherente y lógica con introducción, desarrollo y conclusión claros.	25 %
Dominio del Tema	Demostración de conocimiento profundo de los algoritmos y estructuras implementadas. Comprensión de las decisiones de diseño tomadas	55 %
Uso de Ejemplos y Aplicaciones	Ejemplos relevantes que ilustran claramente el funcionamiento del sistema.	10 %
Calidad Visual	Diseño limpio y profesional del PDF: figuras, tablas, formato y presentación general.	10 %

Rúbrica de evaluación presentación

Criterio	Descripción	Peso (%)
Dominio del Tema	Demostración de conocimiento profundo y comprensión de conceptos avanzados. Capacidad de explicar el trabajo, y responder preguntas con seguridad.	60 %
Uso de Ejemplos y Aplicaciones	Ejemplos relevantes que ilustran claramente los conceptos.	10 %
Estructura y calidad visual	Diseño limpio, legible y profesional de las diapositivas o material de apoyo usado. Presentación con un hilo lógico y ordenado	10 %
Habilidades de Comunicación	Comunicación clara y efectiva, buen uso del lenguaje y modulación de la voz.	10 %
Gestión del Tiempo	Ajuste al tiempo asignado, ritmo adecuado.	10 %